

Craft + Goal in Five Minutes

How to give an AI a memory, and then put it on autopilot — safely

A quickstart for people who don't know the tool yet, and don't need to know much about AI to start.

1. The strange problem nobody warns you about

Here's something odd about today's AI coding assistants. They are genuinely brilliant in the moment — and they have almost no memory.

Close the window, open a fresh one tomorrow, and the assistant has forgotten what you decided yesterday: which approach you rejected, which bug is still open, what "done" even means for this project. It's like working with a world-class expert who has amnesia every morning. Every day you re-explain the whole expedition before any real climbing happens.

So the first question isn't *"how do I make the AI smarter?"* It's *"how do I give the project a memory the AI can read?"*

That's exactly the job of the two tools in this guide.

The one analogy, which we'll carry the whole way: think of your project as an **expedition**. - **Craft** is the expedition's **logbook** — the durable record of where you are, what's blocking you, and what's next. - **Goal** is the **autopilot** — it reads the logbook and keeps moving toward the summit, but it stops at every toll booth and asks you before doing anything risky. - A **controller agent** (more on this at the end) is the **expedition leader** who can run the whole thing on your behalf.

Keep that picture in your head. Everything below is just detail hung on it.

2. Craft — the logbook that gives your project a memory

Craft is a small tool that keeps a *ledger*: a plain, structured file that records the state of your project so it survives between sessions.

You don't have to learn a database. The logbook is two files:

- `.craft/ledger.yml` — the **source of truth** (a structured text file the computer reads).
- `CRAFT.md` — a **human-readable view** of the same thing, with links you can click.

What goes in the logbook? Just the things you'd otherwise keep re-explaining. Each is a "row" with its own ID so anything can point at anything else:

- **Contexts** — the areas of work (the whole expedition, and sub-camps within it).
- **Blockers** — what's stopping progress, *and the condition that would unblock it*.
- **Decisions** — open questions and, once chosen, the answer plus *why*.
- **Gaps** — known holes you haven't filled yet.
- **Definitions** — what your project means by its own words.
- **Next moves** — the single next action for a context.

That's the whole idea: instead of the plan living in your head (and evaporating overnight), it lives in a logbook that both you *and* the AI can read tomorrow. The logbook is the memory.

One honest rule worth knowing early: Craft won't let you secretly "resolve" a raw blocker by waving your hands. A blocker has to be *refined* into a real closure condition before it can close. The logbook is designed to keep you

honest, not just organized.

3. Goal — the autopilot that knows when to stop

A logbook is useful on its own. But the magic happens when something *reads* it and acts.

That's **Goal**. Goal is a control loop — the same idea as a thermostat or a plane's autopilot. It runs a simple cycle, over and over:

1. **Read the frontier** — look at the logbook and list what's actually ready to do.
2. **Classify the risk** of each item — is this safe, or is it dangerous?
3. **Do the safe work** — hand each safe item to whichever tool owns it.
4. **Stop at the toll booth** — for anything risky, *stop and ask you first*.

Step 4 is the whole point, so let me say it plainly. Goal is **fail-closed**. That's a safety term borrowed from engineering: when a fail-closed system isn't sure, it *stops* rather than guesses. If a task would change real files, run shell or network commands, publish something, or commit code — or if Goal simply can't tell how risky it is — it **does not do it**. It pauses and waits for your explicit "yes."

So the headline is not "Goal automates everything." The honest, more useful headline is:

Goal does the safe, boring work automatically — and the clever part is that it knows exactly when to stop and ask you.

That's what makes it safe to leave running. It can't quietly wreck your project, because the risky moves are gated behind your approval, every time.

4. Snapping them together: the easy pipeline

Now put the two halves of the analogy together and you get a tiny, hands-off-but-safe pipeline:



The loop is: **the logbook holds the state, the autopilot moves it forward, and you only get pulled in for the decisions that genuinely need a human.** You stop being the project's memory and the one re-explaining everything, and become the person who approves the important turns.

That's the "automated pipeline" — not a black box that does whatever it wants, but a patient loop that handles the routine and escalates the rest.

5. The 5-minute recipe

Enough theory. Here's the smallest real run. (These are conceptual steps — your AI agent, like Claude Code or Codex, runs the actual Craft operations for you when you ask in plain language. You don't need to memorize commands.)

Step 1 — Start the logbook. Tell your agent:

"Start a Craft project here for . Purpose: ."

It creates `.craft/ledger.yml` and `CRAFT.md`, with your first context and a first next move.

Step 2 — Write down where you are. Add a blocker, a decision, or a gap — whatever is actually true right now:

"Add a blocker: the login page has no error handling. It's unblocked when failed logins show a message."

Step 3 — Set the next move. Name the single next action:

"Set the next move: add error handling to the login form."

Step 4 — Hand it to the autopilot. Now ask Goal to drive:

"Run the Goal loop toward: a working login page with error handling."

Goal reads the frontier, sees the safe authoring work, does it, and **stops** the moment it wants to do something risky — like committing the change — to ask you. You review, you say yes, it continues.

Step 5 — Read the logbook anytime. Ask "what's the Craft status?" and you get the current blockers, decisions, gaps, and next moves — the project's memory, intact, ready for tomorrow's session.

That's the entire pipeline. Five steps, and most of them are one sentence.

6. Going further: the conductor (a controller agent)

So far *you* are talking to the AI. The next step up is to let an agent be the **expedition leader** — the conductor of the orchestra — so it talks to the other agents for you.

Here's the picture. A **controller agent** is an AI whose job isn't to write the code itself, but to *direct* the work: it reads the Craft logbook, decides what should happen next, and then calls the right worker to do it. The workers can be:

- **Claude Code** or **Codex** — coding agents that actually edit files and run the Goal loop;
- **itself** — the controller can also just do a step directly when that's simplest;
- **other models** entirely — the controller can call out to whatever model is best for a task.

Two concrete ways people set this up:

- **A Hermes-class model as the controller.** "Hermes" here means an open-weight model running as the brain that issues the instructions — the messenger that carries work between you, the logbook, and the coding agents.
- **An OpenClaw agent runtime.** OpenClaw is an external *agent runtime* — think of it as a stage the controller stands on, with a proper way to start an agent, send it a turn of work, wait for the result, and cancel it if it gets stuck. In this project, the recommended way to plug it in is its **gateway** (a request-and-response interface) when you need real agent lifecycle control, or a simple **command-line call** for quick one-shot jobs.

The shape is always the same, and it's just our analogy one level up:

```
You → Controller agent (the leader)
      | reads the same Craft logbook
      ├── Claude Code (worker)
      ├── Codex (worker)
      ├── itself (worker)
      └── another model (worker)
```

Everyone reads from the **same logbook**, so nobody loses the plot — and the same toll-booth rule still holds: risky moves stop for human approval. The controller makes the expedition run with less of your attention; it doesn't remove the safety gate.

(One honesty note: the controller pattern is the natural next step, and OpenClaw is modeled in this repo as an external agent runtime. Treat this section as the design you grow into, not a button that already exists — see the source sheet.)

7. What it will *not* do for you (read this part)

Because trust matters, here's the plain list of limits — these are features, not gaps:

- It **won't** make risky changes silently. File edits at scale, shell/network commands, publishing, and commits all **stop for your approval**.
- It **won't** guess when it's unsure. Unknown risk is treated as risky, so it stops.
- It **won't** invent that work is finished. Closing a blocker needs a real condition; finishing a step needs real evidence.
- It **won't** keep the plan only in the AI's head. The plan lives in the logbook, which is why tomorrow's session still knows what today's decided.

The point of all of this isn't to replace your judgment. It's to **carry the project's memory and do the routine work**, so your judgment goes to the few decisions that actually need it.

8. Install it in your own project (self-contained)

Here's the part that makes this a *tool*, not just a story: Craft and Goal install on their own. **You do not need the whole framework they came from.**

Concretely, two folders go into your project's "skills" directory (the place your AI agent looks for its abilities — `.claude/skills/` for Claude Code, `.agents/skills/` for others):

```
your-project/
  .claude/skills/
    craft/      # the logbook    – no code dependencies at all
    goal/       # the autopilot – plain Python 3, standard library only
```

The honest dependency list is short:

- **Craft** needs **nothing** — your agent just reads and writes plain files.
- **Goal** runs on **Python 3** out of the box. The *only* third-party package anywhere in the kit is `jsonschema`, and only if you want to run Goal's self-check (`pip install jsonschema`).
- Goal mentions other helpers (for approvals, for bounded tasks). Those are **optional** — if they're not there, Goal just does its fail-closed thing and stops to ask you. You lose nothing you actually need.

If your package came with `install.sh`, the whole thing is one line:

```
./install.sh /path/to/your/project
```

That copies the two folders in, checks Python, offers to add `jsonschema`, and runs Goal's self-check (you should see `goal-fixtures-pass`). Full details and the manual route are in `INSTALL.md`.

9. Your single next action

Don't try to adopt everything. Do just this:

Open your project and tell your AI agent: **"Start a Craft project here."**

That one command gives your project a memory. Everything else in this guide grows from there.

See `source-trace.md` for exactly where each claim comes from. This is a learning guide, not the source documentation — when in doubt, the linked skill files are the authority.